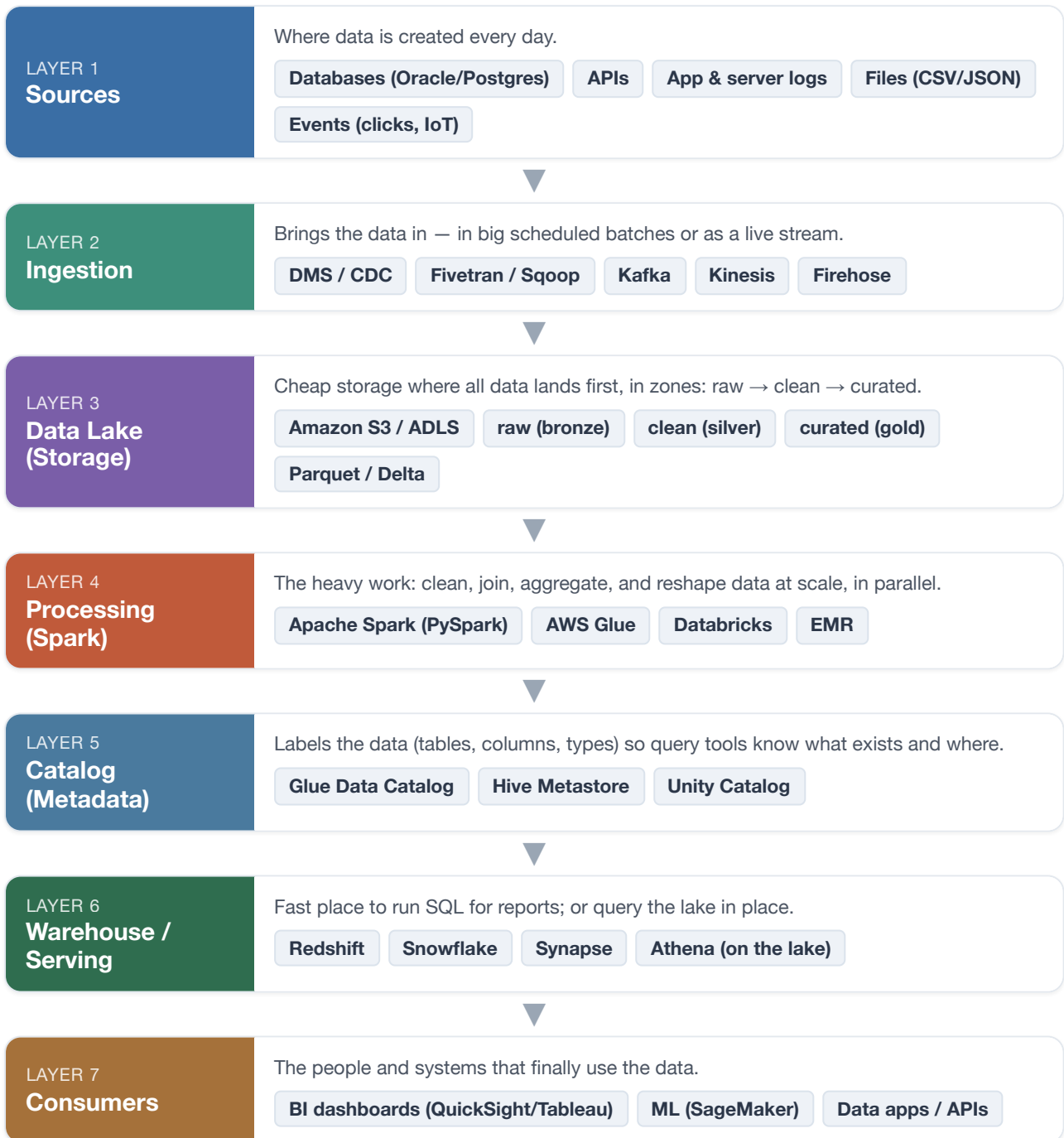


Data Engineering Architecture

How data flows from where it's created to dashboards and AI — every component explained

The architecture at a glance



RUNS ACROSS EVERY LAYER (THE SUPPORT SYSTEMS)

Orchestration — schedules & sequences the whole pipeline, with retries. Airflow, Step Functions, Glue Workflows.

Governance & Security — who can see what, encryption, PII. IAM, KMS, Lake Formation.

Data Quality — tests that stop bad data before it reaches users. dbt tests, Great Expectations.

Monitoring & Cost — alarms on failures, latency, and spend. CloudWatch, Spark UI, logs.

Infra as Code & CI/CD — the platform is defined as code and deployed safely. Terraform, Git, Jenkins.

The Data Engineer — designs, builds, and runs all of the above so clean data is delivered every day.

What each component does (plain language)

1. Sources — where data is born

Every action a company records: an order placed, a button clicked, a payment made, a sensor reading.

DE's job: connect to these systems and pull the data out reliably without slowing them down.

2. Ingestion — bringing the data in

Some data arrives on a schedule (a nightly copy = **batch**); some arrives continuously, event by event (a live feed = **streaming**).

DE's job: set up the pipes (DMS for database changes, Kafka/Kinesis for live events), handle retries, and make sure nothing is lost or duplicated.

3. Data Lake — the big cheap storage room

All data lands here first, exactly as it arrived, in a very cheap store (Amazon S3). It's organized into zones: **raw** (untouched), **clean** (fixed up), **curated** (ready to use).

DE's job: design the zones and folder layout, choose file formats (Parquet), and partition data by date so it's fast and cheap to read later.

4. Processing with Spark — the heavy lifting

This is where messy data becomes useful data: cleaning, joining tables, and adding things up. Because there's too much for one machine, **Spark** splits the work across many machines that run at the same time.

DE's job: write the transformations (in PySpark), and tune them so big jobs run fast and cheap (partitioning, joins, avoiding bottlenecks).

5. Catalog — the labels on the shelves

Once data is processed, we record what each table is and what columns it has, in one central place. Now any query tool can find and understand the data.

DE's job: keep the catalog up to date (via crawlers or definitions) so analysts can just write SQL.

6. Warehouse / Serving — where questions get answered

The clean, organized data is put somewhere built for fast questions — a **data warehouse** (Redshift/Snowflake) — or queried directly on the lake with Athena.

DE's job: model the data into easy-to-query tables and load it efficiently so dashboards respond in seconds.

7. Consumers — the people who use it

Finally, the data powers **dashboards** (charts business teams look at), **AI/ML models**, and apps.

DE's job: make sure the data behind them is correct, fresh, and trustworthy.

In one line: data is created (sources), brought in (ingestion), stored cheaply (data lake), cleaned and combined at scale (Spark), labeled (catalog), served for fast questions (warehouse), and finally used in dashboards and AI (consumers) — with scheduling, security, quality checks, and monitoring running across the whole thing.